The Symbol Game: A First Game with S(n), MSS-N, and Basic Packets

Jeremy H. Carroll

2026-10-02 · Version 0.1.0 · Introductory preprint · Author-review edition

Code baseline: 3f3399766108adc4c4e7615dce2573a57be57dd6

Abstract

The Symbol Game separates proposing a change from checking that change. This introduction uses only the Genesis stages S(n), the Minimal Symbol System atlas MSS-N, and basic packets. A reader can play a first game by declaring a small collection of marks, requesting a distinction, proposing a relation, and asking an independent checker to test a stated rule. A packet records the proposal; a receipt records the check and its limits. We distinguish a Genesis stage from an atlas entry and from an execution of the game. Eight gradual examples teach ground counts, observational separation, compact notation, consistent renaming, typed prompts and answers, scoped returned packets, bounded sessions, and extension round trips. Self-contained code and a Jupyter workbook make each example replayable without claiming to implement Genesis or the full game. The appendix previews the advanced mathematical and practical routes, which require additional definitions, evidence, and checkers.

1. Start with a question

Imagine two marks, a and b, on a card. You retain both marks but have not yet said how a reader can distinguish them. Your task is to supply a distinction and make its checking rule visible. A player proposes a relation. A referee checks the relation against the marks and the agreed rule. The referee writes what passed, what failed, and what was not tested.

That is enough to begin the Symbol Game. A player can be a person, a program, or a language model. A referee can be another person following a finite rule or a deterministic program. A program is useful when the rule is precise enough to replay. Eloquence does not replace a check, and an accepted packet does not by itself establish every claim in the player's explanation.

There are indefinitely many possible instances, representations, goals, and strategies. The first lesson needs only one finite instance. The advanced game becomes demanding once its checking rules involve serious mathematics, software, experiments, or combinations of them.

2. Three objects to keep distinct

S(n) names a Genesis derivation stage. New structure must be justified by an unresolved Genesis obstruction. An external mathematical interpretation may describe a stage; it cannot create the next stage. This paper summarizes the repository's core stages and does not supply a new derivation.

MSS-N names an entry in an externally designed atlas of representations and checking obligations. Its index labels a specification. A listed next entry is not a proved transition to that entry. A proposed transition and a checked, licensed transition are different records.

The Symbol Game is the proposal-and-check workflow applied to a declared instance. It can examine a Genesis claim or an MSS obligation, provided the relevant representation and checking

rule are supplied. Running the workflow does not equate their indices. In particular, MSS-2 is not S(2).

Here is the small part of the Genesis core needed to orient a beginner:

Stage	Reading for this introduction
S(0)	Ground: the presentation `<>` names an empty carrier. It contains no admitted mark.
S(1)	An externally supplied mark may be admitted. Bare supply alone changes no derivation.
S(2)	Marks are retained, but without distinguishing observations they remain observationally collapsed.
S(3)	A demand for separation requires a witnessed distinction. Directed cuts belong here, not to S(2).
S(4)	The declared finite separation task closes when its required distinctions are checked.
S(5)	External continuation introduces a new demand rather than automatic progress.
S(6)	Extension must preserve an explicit way to return to the predecessor.

This table is an orientation, not a replacement for the core definitions and their proof records. Completion of one finite task does not mean all possible future tasks have been solved. The number of currently unresolved obstructions is also different from the persistent history of checked and refused events.

The corresponding beginner atlas route starts at MSS-0 (ground), MSS-1 (an atomic mark), and MSS-2 (relational pairings). A post-hoc comparison of an atlas representation with a Genesis state is a crosswalk, not evidence that either generated the other.

3. Agree on the board before playing

For our first game, the carrier is exactly `a, b`. The goal is to provide a directed relation from a to b. The permitted relation must name only those two marks, use the declared relation label, and never relate a mark to itself. The direction is part of this example's declared task. Irreflexivity alone would not prove that every pair is separated.

Agree on three things before proposing a move:

1. Which marks and relation label are admitted?
2. What finite property will the referee check?
3. How many attempts are available, and what happens when they are exhausted?

A failed proposal leaves the accepted board unchanged. A stopped attempt is recorded as unresolved if no checked answer was obtained. A new mark requires a new declaration or a separately checked extension; it cannot be smuggled into a repair of the current board.

4. A basic packet and a receipt

A packet is a container for a proposal. We use the following teaching format:

```
{
  "schema": "symbol-game-basic-packet/v1",
  "carrier": ["a", "b"],
  "relation_label": "separates",
  "goal": ["a", "b"],
  "candidate": ["a", "separates", "b"]
}
```

This is a deliberately small teaching schema, not a native runtime wire contract. The examples `<>`, `<a>`, and `<ab>` elsewhere in the atlas are presentation forms; the JSON fields here are a convenient way to write a finite checking exercise. Neither notation licenses a Genesis extension.

The referee tests the schema, uniqueness of the carrier marks, membership of each endpoint, the relation label, absence of a self-relation, and agreement with the declared goal. It returns a receipt such as:

```
{
  "status": "CHECKED",
  "property": "declared-directed-separation",
  "scope": "symbol-game-basic-packet/v1"
}
```

The receipt means that the candidate satisfies this finite teaching rule. It does not certify the semantics of the English word "separates," establish all Genesis obligations, prove an MSS lift, or authorize an external action. The packet, the checking rule, and the receipt should be kept together so that someone else can repeat the check.

5. Play and change one thing

First predict the referee's result without running it. The packet above passes. Change its candidate to `a separates a`: the referee refuses a self-relation. Change the second endpoint to c: the referee refuses an endpoint outside the declared carrier. Reverse the candidate to `b separates a`: it is irreflexive, but does not meet this directed goal. Remove the relation label: the packet is incomplete.

Each refusal teaches a different rule. Repair the candidate within the same board and try again. If the desired change concerns the carrier or goal, declare a different instance and preserve the original receipt. Do not turn an altered question into an apparent solution of the old one.

The companion command `python code/basic_packets.py` prints these outcomes and exits unsuccessfully if a negative control unexpectedly passes. Its checker uses only finite strings and lists. It does not call an engine, provider, or network service and makes no claim about advanced runtime functionals. The local tests also exercise malformed inputs and a missing goal. These checks support the teaching example, not a completeness claim about the full game.

6. Reading the compact marks

The "hieroglyphics tricks" are ways of writing a declared structure compactly. They help a reader see boundaries and reuse a pattern. The meaning comes from an explicit grammar and task, not from the appearance of the glyphs.

Start with the opaque whole `<>`. Although its displayed spelling uses two characters, that does not supply two admitted objects, a direction, or a relation. In the ground exercise there is one presentation and zero admitted marks. Next, `<a>` is a supplied whole with a mark. A supplied `<ab>` still needs a declared interpretation before its typography can stand for a directed pair. The Genesis primitives audit makes this boundary explicit.

For the following atlas-style illustrations, declare that `<ab>` means the directed pairing from a to b and that `<<ab><>>` records an obstruction attached to that pairing. Under this convention:

Compact display	How to read it in this declared exercise
`<>`	Ground presentation or primitive-difference token, according to the stated register
`<a>`	A supplied atomic mark
`<ab>`	Directed pairing from a to b, when that interpretation is supplied
`<ba>`	The reversed directed pairing
`<<ab><ba>>`	A container showing both declared directed pairings
`<aa>`	A self-pair; refused by the declared irreflexive relation rule
`<<ab><>>`	A declared pairing with an attached obstruction marker

These are reading examples from the atlas, not a new universal parser. The companion uses explicit JSON lists for the actual relation checks. Its result does not establish that every display above is legal in every wire dialect. The older MSS curriculum includes a different Grammar-000 register with application and dispatch syntax. Record its register before comparing it with the later atlas. A spelling that parses in one regime may remain inactive at another stage.

Three practical habits make compact notation useful:

1. **Declare the reading.** Say whether a token names a mark, a relation, an obstruction, or a container. A change of reading changes the problem.
2. **Separate boundaries from contents.** Keep the representation of a packet distinct from the marks admitted into its carrier.
3. **Rename consistently.** Replacing a by u and b by v throughout a declared relation preserves the shape a $\to$ b as u $\to$ v. Replacing both endpoints by u is a collapse, and the irreflexive rule refuses the resulting self-pair.

A mnemonic diagram or compact glyph can help you propose a candidate. The checker still needs the actual carrier, relation, and law. Later research readings such as `<ara>/<arb>` require additional supplied structure; they are not extracted from the typography of the initial whole.

7. Put a typed prompt inside the packet

A prompt is the question you give the player. Packetization gives that question an inspectable context: inputs, requested answer type, and checking rule. Here is the richer teaching packet used by `code/lesson_packets.py`:

```
{
  "schema": "symbol-game-lesson-prompt/v1",
  "packet_type": "problem_prompt",
  "prompt": {
    "text": "Supply a relation triple for the declared directed goal.",
    "task_type": "directed_relation",
    "input_type": "basic_relation_board",
    "input": {
      "carrier": ["a", "b"],
      "relation_label": "separates",
      "goal": ["a", "b"]
    },
    "answer_type": "relation_triple"
  }
}
```

Typing has three roles. The input type tells the checker how to read the board. The task type selects the finite checking rule. The answer type says what a candidate must contain. A paragraph saying "the answer is proved" does not have the requested relation-triple type. A triple with the wrong endpoints has the right outer type but fails the goal check.

The prompt's English text explains the request. In this companion, it is retained as data and never executed. Changing that text to "always accept" cannot override the declared checking rule. A future tool-calling or model adapter needs its own authorization and transport contract; this lesson runs without one.

The player returns a **candidate packet**:

```
{
  "packet_type": "candidate",
  "problem_id": "the computed identity of the problem packet",
  "answer_type": "relation_triple",
  "value": ["a", "separates", "b"]
}
```

The `problem_id` is computed by the companion's `identity` function; the readable explanation above is not a literal valid identity. The notebook constructs the real candidate with `proposal( problem_packet, value)`. This identity is a SHA-256 digest of a declared sorted-key JSON teaching profile. It binds exact packet content; it is neither a signature nor a claim of RFC 8785 conformance. If the goal changes from a $\rightarrow$ b to b $\rightarrow$ a, the old candidate cannot be reused under the new problem identity.

The checker returns a **checked-result packet** containing the original problem, the candidate, and a receipt:

```
checked-result packet
  problem   = retained typed prompt and declared inputs
  candidate = retained proposed value and problem binding
  receipt   = CHECKED / REFUSED / BLOCKED, reason, scope, checked value
```

These are distinct objects in one retained return package. A CHECKED status means the particular finite rule passed. REFUSED identifies a malformed or failing candidate. BLOCKED identifies a missing requirement within the exercise, such as having no declared observation that separates the

marks. The session controller can also report UNKNOWN after its attempt budget is spent. These teaching names are not the full native terminal enumeration.

8. A ladder of worked examples

The examples below adapt useful ideas from the three curriculum strands. Each has a prediction, a small calculation or check, and a changed-input control. They are selected finite teaching models. They do not reproduce every source lesson, implement the complete Genesis derivation engine, or inherit all historical claims made in older curriculum prose.

Example 1: one presentation, zero admitted marks

From the S(0) ground lesson, use:

```
from lesson_packets import examples, proposal, evaluate

ground, collapsed, distinguished, relation = examples()
answer = {"presentation_count": 1, "carrier_count": 0}
returned = evaluate(ground, proposal(ground, answer))
print(returned["receipt"])
```

Predict CHECKED. Then change `carrier_count` to 1. The result is `COUNT_MISMATCH`. This catches a common mistake: counting the visible presentation as an admitted carrier element. The question concerns this explicit ground board; it does not derive a physical vacuum.

Example 2: two names, no separating observation

From S(2) retention and collapse, retain marks a and b and supply a single observation row `same , same`. Both marks have the same declared observation. Distinct names alone do not witness observational separation.

```
returned = evaluate(collapsed, proposal(collapsed, 0))
print(returned["receipt"]["reason"])
# NO_DECLARED_SEPARATOR
```

The result is BLOCKED. It concerns the finite supplied observation table, not all possible observations or a proof that separation is impossible.

Example 3: supply and check a separator

The next exercise supplies a second observation row `left, right`. The candidate answer is its index 1:

```
returned = evaluate(distinguished, proposal(distinguished, 1))
print(returned["receipt"]["checked_value"])
# 1
```

Index 0 still fails because its row is `same, same`. This teaches the S(3) separation demand using a declared observation model. Supplying the new row is an explicit change of input; the previous BLOCKED receipt remains a record of the previous board. The teaching model itself licenses no Genesis advance.

Example 4: a relation witness and its law

From the MSS relational examples and Genesis primitive-difference lessons, require a supplied directed witness and an irreflexive rule:

```
valid = evaluate(relation, proposal(relation, ["a", "separates", "b"]))
invalid = evaluate(relation, proposal(relation, ["a", "separates", "a"]))
print(valid["receipt"]["status"], invalid["receipt"]["reason"])
# CHECKED SELF_RELATION
```

The relation, witness, and law are supplied inputs. The endpoint separation comes from checking those inputs. Writing a more elaborate glyph does not supply the missing witness. Reversing the triple gives another valid outer shape but fails the particular directed goal.

Example 5: change names while preserving structure

From the Genesis renaming lesson, rename the entire carrier consistently:

```
from lesson_packets import rename_edges

marks, edges = rename_edges(["a", "b"], [("a", "b")],
                            {"a": "u", "b": "v"})
print(marks, edges)
# ['u', 'v'] [('u', 'v')]
```

The helper requires a bijection on the declared carrier. Change the map to `a` → `u`, `b` → `u` and it raises a refusal error. This establishes preservation of this finite directed edge under this renaming; it does not implement a canonicalizer for every native Genesis regime.

Example 6: return a useful refusal, repair, and replay

Ask the relation checker about `a separates a`. The return packet preserves both the request and the refused answer, with reason `SELF_RELATION` and no checked value. Repair the answer to `a separates b` under the same prompt. The returned checked value is the relation triple, which a caller can consume only after checking the receipt and its scope.

```
from lesson_packets import replay_return

assert replay_return(valid)
changed = evaluate(relation, proposal(relation, ["a", "separates", "a"]))
changed["receipt"]["status"] = "CHECKED"
assert not replay_return(changed)
```

Replay checks the retained problem and candidate again. Relabeling a refusal cannot turn it into a passing check. This is a local consistency test, not producer authentication: an attacker who replaces every object can construct a different self-consistent exercise. Preserve the expected problem identity when using a return from an untrusted source.

Example 7: a finite attempt budget

A strategy proposes a self-pair and then a valid directed pair. With one attempt available, it stops UNKNOWN. With two attempts, the second candidate passes, and the first refusal is still retained:

```
from lesson_packets import bounded_session

answers = [["a", "separates", "a"], ["a", "separates", "b"]]
print(bounded_session(relation, answers, fuel=1)["status"])
print(bounded_session(relation, answers, fuel=2)["status"])
# UNKNOWN
# CHECKED
```

This explains the value of bounded sessions and honest stopping conditions. A longer budget helped this particular scripted sequence; the example makes no general claim about convergence, cost savings, or strategy quality.

Example 8: keep a way back when extending

From the S(6) descent and MSS up/down lessons, retain an explicit predecessor representation. In a small declared illustration, encode old marks as `("old", mark)`. The down-map recovers an admitted old mark; a new term returns `NON_DESCENT`:

```
old_marks = {"a", "b"}
def up(mark):
    return ("old", mark)
def down(term):
    return term[1] if term[0] == "old" and term[1] in old_marks else "NON_DESCENT"

assert all(down(up(mark)) == mark for mark in old_marks)
assert down(("new", "c")) == "NON_DESCENT"
```

This demonstrates a finite round-trip obligation. It supplies neither a new Genesis stage nor a licensed MSS lift. A real extension needs its own source regime, maps, preservation properties, and licensing record.

9. The possible stages of a play session

A play session has a lifecycle. Its workflow stages are separate from the S(n) derivation stages and MSS-N atlas indices.

Session stage	What you do	What can happen next
Declare	Fix the carrier, task, schema, checker, and budget	Begin when the required inputs and checker exist
Packetize	Put the explanation, typed inputs, and requested answer type in a problem packet	Refuse an invalid prompt; retain a valid request
Propose	Supply a candidate bound to that request	Check it while treating its explanation as a proposal
Check	Validate types and bindings, then the declared finite property	Return a checked value, a typed refusal, or a blocked obligation
Repair	Change a candidate within the same board	Recheck; keep the earlier refusal

Session stage	What you do	What can happen next
Close or stop	Accept the scoped result, or record an unresolved bounded stop	Replay or compare retained records
Request extension	Declare a changed carrier, observation set, or capability	Check additional obligations; no automatic advance
Replay and learn	Recompute the check from the retained objects	Reuse a scoped result or reproduce a concrete defect

Some instances close after one candidate. Others require repeated repairs, branching search, or a new checker. A session may halt because the answer was checked, because a required capability is absent, or because its fuel was spent. These are different outcomes. Closing a local task does not close the whole Symbol Game.

10. The value of the returned packet

A useful return gives the caller something concrete to do. In our examples, it gives the exact checked count, the index of a separating observation, or the checked directed relation. A refused return points to a repairable field. A blocked return identifies an additional input or obligation. An UNKNOWN session preserves the attempts without making a conclusion it did not check.

The packet also makes the question portable: another reader can see what was asked, which value was proposed, and which rule was applied. A Jupyter notebook can display that sequence; a program can branch on its typed status. A digest helps detect a changed question. A preserved refusal makes a changed-input exercise reproducible. None of these alone proves that a natural-language interpretation is correct.

There are three different values to discuss:

- **Returned data:** the candidate and any scoped checked value.
- **Evidence value:** what the reproduced check supports about those data.
- **Practical value:** whether this checked property is useful for your real task. That depends on choosing a meaningful carrier, question, and checker.

For instance, the relation checker gives no useful guarantee about a medical, legal, or scientific decision merely because its JSON is valid. A domain task needs the right domain predicate and evidence. This is why later papers introduce stronger certificates and careful limits.

11. What you have learned

You can now declare a finite board, name its goal, propose a packet, predict the referee's check, and interpret a scoped receipt. You can also distinguish an S(n) derivation from an MSS-N specification. These are sufficient entry skills. Advanced algebra or optimization is unnecessary for this first game.

The next step is to choose a useful play route. Begin with changed-input checks and packet repair. Move to graph repair, proof checking, or experimental comparison only after reading the additional rules for that route. A finite check can be strong evidence for its declared property while leaving broader questions open.

Mechanism inventory

The manuscript is introductory exposition. The companion includes finite schema-and-relation checks, typed prompt and answer bindings, observation-table checks, bijective renaming, bounded sessions, and returned-packet replay with positive and negative controls. The S(n) table summarizes existing core documentation; the MSS descriptions summarize atlas specifications. Neither companion code nor exposition derives new Genesis stages, implements the native host, or proves an advanced lift.

A practical reading and notebook path

Start with the Paper 000 workbook. It executes the eight examples above, prints actual typed prompt, candidate, and return packets, and asks you to predict changed-input outcomes. Run it from the package root or its notebook directory. All inputs are local and all checks have a finite teaching scope.

The Landau companion provides a learner guide, an examination, and a separate worked key in _lev_landau_theoretical_minimum_quiz/. Its core questions follow the eight examples. Four optional final challenges supply additional finite definitions and do not license a Genesis or MSS transition.

Continue with Paper 001 for a stronger worked certificate and Paper 005 for practical session management. Papers 002 and 008 develop representations, search, and extension obligations. The early papers should be read as a progressive route: understand the question and scoped return first, then add the mathematics needed for a stronger claim.

Curriculum source	Idea used here	Scope retained
S(n), `000_sn_core`, lessons 1, 3, 4, and 7	Ground counts, collapsed profiles, separation, descent	Finite teaching representations of core concepts; no new derivation
MSS-N, `600_mssn_frontier`, early ground, pairing, and up/down lessons	Syntax versus stage activation, declared pairings, round-trip checks	Registers must be identified; an example does not license ascent
Genesis, `985_genesis_primitives`, lessons 2, 4, and 11	Supplied relations, refusal boundaries, consistent renaming	Selected audited models, not a complete or faithful extraction of Genesis
Genesis primitives adversarial audit, correction of 2026-10-01	Opaque-whole and attribution boundaries	The relation and its law do not follow from typography

References

The cited curriculum lives in repository dbccpage/the_symbol_game. Its conceptual source baseline is commit 3f3399766108adc4c4e7615dce2573a57be57dd6 (the prefix previously written `3f3399766`). That pin names the documents below. It is separate from any later candidate checkout used to execute this package, and it is not a digest of the files shipped here. Exact hashes of the executed package files are in docs/release_validation_2026_10_02_review_repairs.json. These links identify the cited bytes. This deposit does not re-audit those unbundled documents.

- Carroll, J. H. *S(n) core stages S(0)–S(6)*, `dbccpage/the_symbol_game` at the source baseline above. Definitions of supply, retention, separation, and extension.

- Carroll, J. H. *Minimal Symbol System N*, the same repository and baseline. Externally designed atlas and separation of indices and licensed lifts.
- Carroll, J. H. *S(n) curriculum*, `000_sn_core`, lessons on ground, retention, required separation, and descent. Repository educational material under the core stages linked above.
- Carroll, J. H. *MSS-N curriculum*, `600_mssn_frontier`. Early finite representations and explicitly distinguished historical registers, under the atlas linked above.
- Carroll, J. H. *Genesis primitives: audited educational models*, correction of 2026-10-01, and the *985_genesis_primitives* curriculum. The adversarial audit supersedes blanket extraction and physical-emergence claims in earlier lesson prose.
- Carroll, J. H. *Proposal Fidelity Protocol paper collection*, Papers 001–010, and the separately retained meron benchmark (Paper 011). Companion studies and bounded evidence for the advanced routes below.

Appendix A. Preview of the full Symbol Game

The full game is a family of declared finite tasks, not one universal board. Its deeper routes require the mathematics and implementation appropriate to the property being checked. This appendix previews those routes without making them prerequisites of the introduction.

Route	Additional work and its limit
Packet iteration	Validate a richer schema, repair a refusal, and replay the same instance. Schema validity alone does not prove a mathematical claim.
Exact graph repair (001)	Compare rational residuals and, where supplied, matching lower-bound certificates. Improving a candidate is different from proving an optimum.
Signed presentations (002)	Check signs, gauges, and bounded search within a fixed presentation. A finite exclusion is not a physical classification.
Failure detectors (003)	Admit a detector against explicit tests and negative controls. A constructed example is not a measured error rate.
Native checking and custody (004)	Pin bytes, invoke the relevant host, and preserve outcomes. Hash equality alone is not semantic correctness.
Practical sessions (005)	Manage repairs, refusal, fuel, and reproducibility. A stopped session may remain unresolved.
Rejected-proposal evidence (006)	Retain and replay counterexamples. A catalog entry does not establish detector efficacy.
Teaching (007)	Predict, execute, and vary examples. A successful exercise does not establish learning transfer.
MSS obligations and lifts (008)	Check context maps, transition records, and retractions. An atlas row does not license its successor.

Route	Additional work and its limit
Adversarial review (009)	Challenge a concrete property with a counterexample. Consensus does not replace the property's checker.
Holonomy and pilots (010)	Check finite transport and interpret retained experiments. Small or answer-leaking pilots support no general model ranking.
Meron benchmark (011)	Check a finite cancellation fixture and distinguish disclosed answers from discovery. The retained pilot is not evidence of unaided discovery.

Further representations include rewriting systems, cochains, higher structures, discrete geometry, quantum information, and bounded computation. Each introduces new obligations. Mathematical realizations and physical interpretations remain distinct from the Genesis core. There is no promise here that every route is implemented, that every atlas entry is attained, or that a common algorithm solves all instances.

The practical breadth comes from choosing a carrier, a goal, a proposal strategy, and a checker. The ten learning routes in the educational manual are a useful editorial selection, not a measured popularity ranking or a complete enumeration of ways to play.